

OOPS ka matlab kya hai?

OOPS = Object-Oriented Programming System

OOPS kya hai?

OOPS ek programming style hai jisme hum real-world cheezon ko "objects" ke roop mein represent karte hain.

Isme data (properties) aur functions (methods) ko objects ke andar organize karte hain, taaki code samajhna, likhna aur maintain karna easy ho jaye.

(

OOPS is a programming style where we model real-world things as "objects".

Each object contains data (called properties) and functions (called methods) that operate on the data.

This helps organize code better, makes it easier to understand, reuse, and maintain.

)

Class and Object –

✓ 1. What is a Class?

- **Definition:** Class ek blueprint ya template hoti hai jisme hum object banane ke liye data (properties) aur functions (methods) define karte hain.

What does a Class contain?

A class mainly contains:

1. **Properties (Attributes):** These are the data or characteristics of the object.
Example: color, model, speed for a car.
2. **Methods (Functions):** These are actions or behaviors the object can perform.
Example: start(), stop(), accelerate() for a car.

```
class ClassName {  
  
    method1() {  
  
        // code  
  
    }  
  
    method2() {  
  
        // code  
  
    }  
  
}
```

◆ Types of Inheritance in OOP (Object-Oriented Programming)

Traditionally, there are **5 types** of inheritance:

Type	Meaning
1. Single	One child class inherits from one parent
2. Multi-level	One class inherits from another, which inherits from another
3. Hierarchical	Multiple child classes inherit from one parent
4. Multiple	One class inherits from multiple parents
5. Hybrid	Combination of multiple types

1. Single →

```
JS Clas_Obj.js > Home > Add
1  class Home{
2      Add(){
3          let a=2;
4          let a1=2;
5          console.log("add "+(a+a1));
6      }
7      Mul(){
8          let a=12;
9          let a1=2;
10         console.log("Mul "+(a*a1));
11     }
12 }
13 class Home2 extends Home{
14     Add1(){
15         let a=2;
16         let a1=2;
17         console.log("add "+(a+a1));
18     }
19     Mul112(){
20         let a=12;
21         let a1=2;
22         console.log("Mul "+(a*a1));
23     }
24 }
25
26 h=new Home2();
27 h.Add();
28 h.Mul();
29 h.Mul112();
30
```

Code → [click me](#)

2 What is Multilevel Inheritance?

Multilevel Inheritance me ek class **dusri class se inherit** karti hai, aur wo class **teesri class se inherit** karti hai.

Jaise ek **chain** banti hai.

```
JS single_1.js JS Multilevel_1.js X
Inheritance > JS Multilevel_1.js > Parent > showParent
1  class Grandparent {
2      showGrandparent() {
3          console.log("I am Grandparent");
4      }
5  }
6
7  class Parent extends Grandparent {
8      showParent() {
9          console.log("I am Parent");
10     }
11 }
12
13 class Child extends Parent {
14     showChild() {
15         console.log("I am Child");
16     }
17 }
18
19 let c = new Child();
20 c.showChild();
21 c.showParent();
22 c.showGrandparent();
23
```

Code →

[Click me](#)

3. Hierarchical Inheritance

One parent class, but **multiple child classes** inherit from it.

```
← → Programme
JS single_1.js JS Multilevel_1.js JS Hierarchical_1.js
Inheritance > JS Hierarchical_1.js > ...
1  class Parent {
2      showParent() {
3          console.log("I am Parent");
4      }
5  }
6
7  class Child1 extends Parent {
8      showChild1() {
9          console.log("I am Child 1");
10     }
11 }
12
13 class Child2 extends Parent {
14     showChild2() {
15         console.log("I am Child 2");
16     }
17 }
18
19 let c1 = new Child1();
20 c1.showParent(); // I am Parent
21 c1.showChild1(); // I am Child 1
22
23 let c2 = new Child2();
24 c2.showParent(); // I am Parent
25 c2.showChild2(); // I am Child 2
26
```

Code -> [Click me](#)

Multiple Inheritance ➡

```
// ✖ ERROR: JavaScript doesn't allow extending multiple classes
```

```
file Edit Selection ... ← → Programme
JS single_1.js JS Multilevel_1.js JS Hierarchical_1.js JS MultipleInheritance_1.js 1 X
Inheritance > JS MultipleInheritance_1.js > C > showC
1 class A {
2     showA() {
3         console.log("Class A");
4     }
5 }
6 class B {
7     showB() {
8         console.log("Class B");
9     }
10 }
11
12 class C extends A, B {
13     showC() {
14         console.log("Class C");
15     }
16 }
17
18 let obj = new C();
19 obj.showA();
20 obj.showB();
21 obj.showC();
22
```

★ Why Error?

- JavaScript only allows one `extends` keyword at a time.
- You **cannot inherit from two classes directly** like `class C extends A, B`.

MIXIN METHOD ➡ :

🔥 What is a Mixin?

Mixin ek aisa technique ya object hota hai jisme hum kuch reusable methods (functions) rakhte hain, aur unhe alag-alag classes me **copy karke** use karte hain.

Why use Mixin?

- JavaScript me **multiple inheritance** nahi hoti.
- Lekin agar tumhe ek class me do ya zyada classes ke behavior chahiye, to tum **Mixin** use karte ho.
- Isse tum ek class me **bahut saare features** add kar sakte ho bina inheritance ke.

```
24
25  CanFly = {
26    fly() {
27      console.log("✈️ I can fly");
28    }
29  };
30  CanSwim = {
31    swim() {
32      console.log("🏊 I can swim");
33    }
34  };
35  CanSwim2 = {
36    swim2() {
37      console.log("I can swim2");
38    }
39  };
40  class Duck {}
41
42  Object.assign(Duck.prototype, CanFly, CanSwim, CanSwim2);
43
44
45  let d = new Duck();
46  d.fly();
47  d.swim();
48  d.swim2();
49
50
```

Code → [click me](#)